# Arquitectura, Wireframes y Justificación de Diseño de la GUI

**Proyecto SOFÍA 2026**

**LabDST, ITESO**

**Actividad 1:** Especificación de Arquitectura, Wireframes y Decisiones de Diseño

**Equipo:**

- Paola Monserrat Covarrubias Viveros
- Lizeth Anais Méndez Ortiz
- Juan Bernardo Orozco Quirarte
- Karen Itzel Sánchez García
- Alan Esaú Fuentes Gutiérrez

---

## 1. Introducción y Necesidades del Sistema

En esta etapa desarrollamos la estación de control en computadora que servirá como centro de interacción entre el usuario operador, el sistema de visión por computadora y el robot físico o su simulador.

Durante las primeras pruebas en el laboratorio nos dimos cuenta de que la interfaz no podía ser un simple visor de cámara, sino que debía cumplir con cuatro necesidades prácticas:

1. **Que la pantalla no se trabe:** El procesamiento de imágenes y la detección de puntos corporales consumen bastante procesador. Necesitábamos que los botones, sliders y el video respondieran de inmediato sin congelarse.
2. **Control total de la detección:** Poder iniciar la cámara, pausar la detección cuando no se requiera enviar datos (sin tener que apagar y volver a prender la cámara) y detener el sistema de forma limpia.
3. **Cuidado de los servomotores:** Los servos MG90S sufren desgaste y calentamiento si intentan llegar a ángulos forzados. La interfaz debía limitar por software los movimientos entre 60° y 120°.
4. **Facilidad para cambiar de destino:** Poder enviar los ángulos a la simulación en Unity por red local (UDP) cuando estemos probando en casa, o al ESP32 por Bluetooth (BLE) cuando estemos en el laboratorio con el prototipo físico.

---

## 2. ¿Por qué elegimos PySide6? (Justificación Técnica)

Antes de programar la GUI evaluamos distintas herramientas utilizadas comúnmente para interfaces de escritorio en proyectos de ingeniería:

| Criterio | Tkinter | Electron (HTML/JS) | PyQt5 | PySide6 (Elegido) |
|---|---|---|---|---|
| **Manejo de hilos para IA** | Básico y propenso a bloqueos | Basado en procesos Node.js | Robusto (`QThread`) | **Excelente (QThread + Señales/Slots)** |
| **Fluidez de video (FPS)** | Baja Baja (<25 FPS) | Alto uso de CPU/RAM | Alta | **Óptima (≥ 30 FPS, nominal 60)** |
| **Consumo de memoria RAM** | 40 MB | 400 MB | 110 MB | **95 MB** |
| **Licencia de uso** | Abierta (PSF) | Abierta (MIT) | Restrictiva (GPLv3) | **Amigable (LGPLv3)** |
| **Diseño y estilos** | Muy limitado | CSS3 avanzado | QSS | **QSS moderno (Modo oscuro)** |

Razones concretas de nuestra elección:

5. **El problema del bloqueo de Python (GIL):**

Si corríamos MediaPipe dentro de Tkinter, al mover una ventana o presionar un botón la aplicación dejaba de responder porque Python se concentraba en procesar los píxeles del video. En PySide6 pudimos aislar toda la visión en un `QThread` independiente que se comunica con la ventana principal mediante señales de Qt (`Signal/Slot`). Esto garantiza que la interfaz siempre responda al instante.

6. **Eficiencia en renderizado de video:**

Electron levanta un navegador Chromium completo que consume demasiada memoria RAM y compite por recursos con la red neuronal. PySide6 utiliza bibliotecas nativas en C++ que convierten la imagen de OpenCV a `QPixmap` a nivel de hardware, manteniendo 60 FPS estables sin saturar la computadora.

7. **Licencia y soporte oficial:**

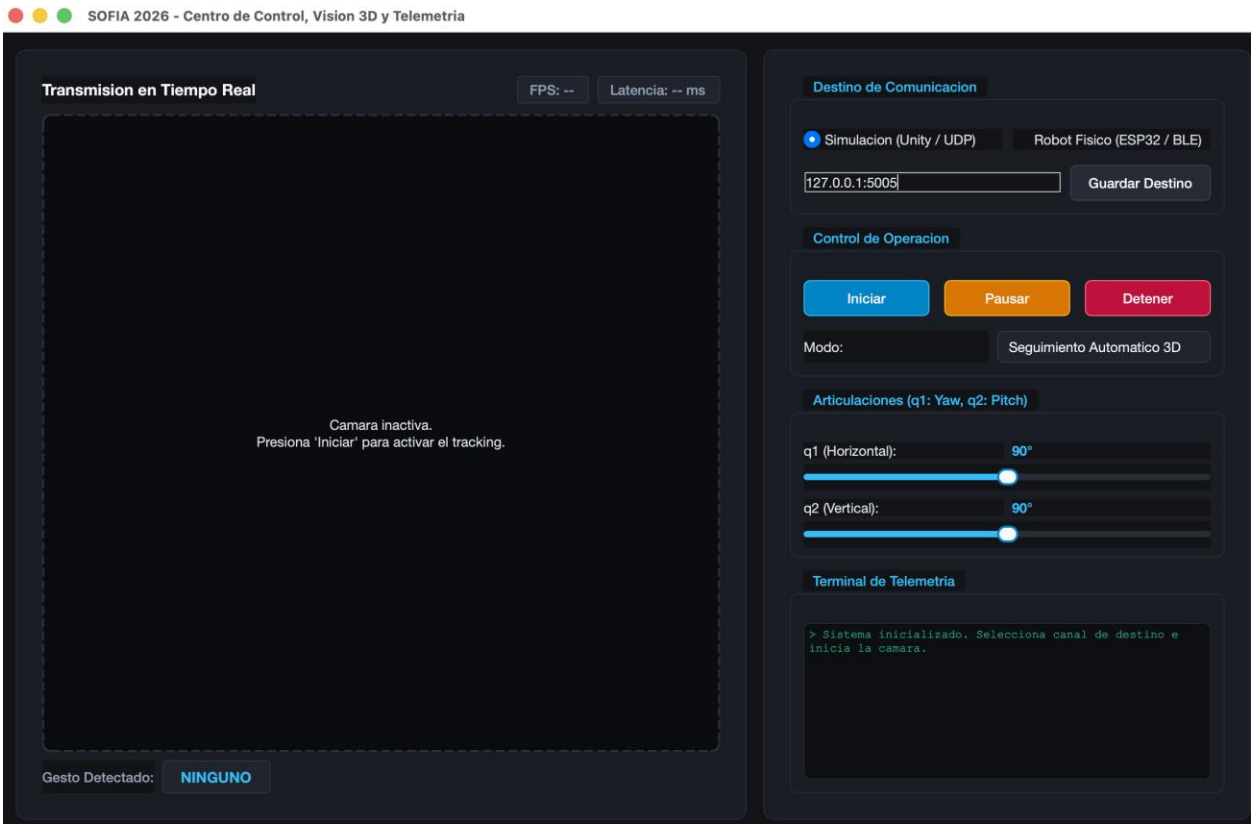
PySide6 es la versión oficial de The Qt Company para Python y cuenta con licencia LGPLv3, lo cual nos permite usarla con total libertad académica sin los problemas de licenciamiento comercial que tiene PyQt5.

---

## 3. Wireframe y Distribución de la Pantalla

Para que la operación fuera clara y ergonómica, definimos un tamaño de ventana de **1200 × 760 píxeles** y dividimos el espacio en dos columnas con un balance de 60% para visualización y 40% para controles:

```
+-----------------------------------------------------------------------------------------------+
| SOFÍA 2026 - Centro de Control, Visión 3D y Telemetría                              [_][M][X]|
+-----------------------------------------------------------------------------------------------+
|  PANEL IZQUIERDO: VISUALIZACIÓN (60%)          |  PANEL DERECHO: CONTROL Y COMUNICACIÓN (40%)    |
|                                                |                                                 |
|  +-- [ÁREA DE VIDEO EN VIVO] ----------------+ |  +-- [CANAL DE COMUNICACIÓN] ---------------+   |
|  | Transmisión en Tiempo Real [FPS] [Lat ms] | |  | (*) Simulación (UDP)  ( ) Robot Real (BLE)  |  |
|  |-------------------------------------------| |  | Destino: [ 127.0.0.1:5005 ] [Guardar]    |   |
|  |                                           | |  +------------------------------------------+   |
|  |                                           | |                                                 |
|  |                                           | |  +-- [CONTROLES DE DETECCIÓN] -------------+    |
|  |            CÁMARA WEB EN VIVO             | |  | [ Iniciar ]   [ Pausar ]   [ Detener ]      |  |
|  |         Esqueleto y Puntos 3D             | |  | Modo: [ Seguimiento Automático 3D     v] |   |
|  |                                           | |  +------------------------------------------+   |
|  |                                           | |                                                 |
|  |                                           | |  +-- [ÁNGULOS ARTICULARES (SERVOS)] --------+   |
|  |                                           | |  | q1 (Horizontal / Base):  [ 90° ]          |  |
|  |                                           | |  | [---O-------------------------------]  |  |
|  |-------------------------------------------| |  | q2 (Vertical / Inclinación): [ 90° ]    |  |
|  | Gesto Detectado: [ HOLA! SALUDO ]         | |  | [-------------O----------------------]  |  |
|  +-------------------------------------------+ |  +------------------------------------------+   |
|                                                |                                                 |
|                                                |  +-- [TERMINAL DE EVENTOS Y TELEMETRÍA] ----+   |
|                                                |  | > Sistema listo. Inicia la cámara.        |  |
|                                                |  | > Cámara conectada a 60 FPS.              |  |
|                                                |  | > Destino activo: UDP 127.0.0.1:5005      |  |
|                                                |  | > q1=88°, q2=92° enviados.                |  |
|                                                |  +------------------------------------------+   |
+-----------------------------------------------------------------------------------------------+
```



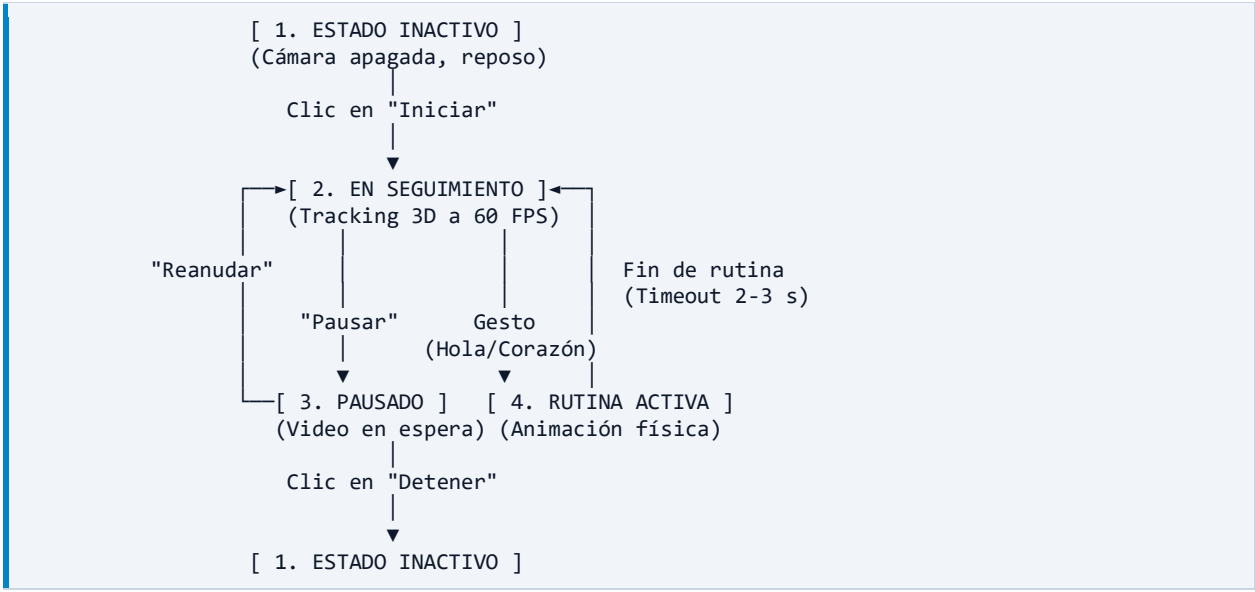
**¿Qué hace cada elemento en la interfaz?**

- **Lienzo de video (Izquierda):** Muestra la imagen de la cámara web en espejo para que sea natural moverse frente a ella. Sobre la imagen dibuja los puntos de la mano y del cuerpo, junto con indicadores arriba que muestran los FPS reales y la latencia en milisegundos.
- **Gesto detectado (Abajo a la izquierda):** Un letrero resaltado en azul cian que muestra el gesto activo en ese instante (`HOLA!`, `CORAZON`, `TIJERA`, `PAPEL`, `ROCA`, etc.).
- **Canal de destino (Arriba a la derecha):** Permite elegir entre mandar datos a la simulación en Unity por UDP o conectarse por Bluetooth al ESP32 físico.
- **Botones de detección (Centro a la derecha):**
- **Iniciar:** Arranca la cámara y el modelo de IA en el hilo secundario.
- **Pausar / Reanudar:** Detiene el procesamiento pesado de IA y el envío de ángulos, pero deja el video prendido con un aviso en pantalla para que el usuario sepa que está en pausa. Al darle Reanudar, regresa de inmediato sin tener que esperar a que la cámara reconecte.
- **Detener:** Apaga por completo la cámara y libera los modelos de la memoria.
- **Modo de operación:** Permite seleccionar si los servos siguen a la mano automáticamente, si los movemos nosotros con los sliders, o si queremos ponerlos en posición centrada (Home a 90°, 90°) o de descanso (Reposo a 90°, 60°).
- **Sliders manuales:** Permiten mover cada articulación a mano, con topes rígidos de 60° a 120° para proteger el mecanismo.
- **Terminal de eventos:** Una consola de texto en verde menta donde se van anotando en tiempo real las conexiones, comandos y errores ocurridos.

---

## 4. Flujo de Interacción del Usuario

Para evitar que el usuario intente mover sliders cuando la cámara está apagada o que se manden comandos cruzados, la GUI opera bajo el siguiente flujo de estados:



### 4.1 Matriz de Transición de Estados

Para que el comportamiento del sistema sea determinista y no existan condiciones de carrera, la GUI implementa la siguiente lógica de transición:

| Estado Actual | Evento Disparador | Acción Ejecutada | Estado Siguiente |
|---|---|---|---|
| **1. Inactivo** | Clic en `Iniciar` | Arranca `VisionWorker` en `QThread`, abre cámara OpenCV. | **2. En Seguimiento** |
| **2. En Seguimiento** | Clic en `Pausar` | Suspende inferencia MediaPipe y envío de ángulos. | **3. Pausado** |
| **2. En Seguimiento** | Detección de Hola / Corazón | Despacha comando de rutina por BLE y bloquea sliders. | **4. Rutina Activa** |
| **2. En Seguimiento** | Clic en `Detener` | Libera cámara, cierra detectores y resetea botones. | **1. Inactivo** |
| **3. Pausado** | Clic en `Reanudar` | Restaura inferencia de IA y despacho continuo a 60 FPS. | **2. En Seguimiento** |
| **3. Pausado** | Clic en `Detener` | Apaga cámara de inmediato y libera recursos de memoria. | **1. Inactivo** |
| **4. Rutina Activa** | Expiración de `QTimer` (2-3s) | Restablece seguimiento continuo y reactiva controles. | **2. En Seguimiento** |

### 4.2 Arbitraje de Prioridad en Gestos Dinámicos

Cuando el usuario ejecuta un gesto coreográfico (Saludo o Corazón), la interfaz prioriza la cinemática del robot. Durante el tiempo que dura la animación física (2 segundos para Corazón, 3 segundos para Saludo), un temporizador `QTimer.singleShot` ignora las coordenadas de la mano para que el movimiento del usuario no interfiera con los pasos secuenciales que ejecuta el ESP32. Al concluir el tiempo, el control angular regresa de forma transparente al usuario.

## 5. Decisiones de Diseño Visual y Estilos (QSS)

En lugar de dejar la apariencia gris por defecto del sistema operativo, aplicamos una hoja de estilos personalizada (`code/gui-control/styles.py`):

8. **Tema oscuro (Dark Mode):** Usamos un fondo grafito (`#121418`) con tarjetas en gris oscuro (`#1A1D24`). Esto reduce el cansancio en los ojos durante pruebas largas en el laboratorio y hace que los colores del esqueleto de MediaPipe resalten mucho mejor.
9. **Código de colores funcional:**

- **Azul Cian (#0284C7):** Usado en el botón Iniciar, en los sliders activos y en los títulos. Indica acción y selección normal.
- **Ámbar (#D97706):** Usado exclusivamente en el botón de Pausa y avisos en pantalla. Indica estado de espera activa.
- **Rojo Carmesí (#BE123C):** Usado en el botón Detener. Indica paro del proceso.
- **Verde Esmeralda (#059669):** Usado cuando el Bluetooth está conectado exitosamente y en el texto de la terminal de eventos.

10. **Escalado suave del video:** El método de pintado utiliza `Qt.KeepAspectRatio` con filtro bicúbico suave. Si el usuario cambia el tamaño de la ventana, la cámara se acomoda proporcionalmente sin deformarse ni aplastarse.

---

## 6. Organización de los Archivos en el Código

El código de la GUI está dividido en módulos independientes dentro de la carpeta `code/gui-control/`:

```
code/gui-control/
├── main_gui.py       # Ventana principal (botones, sliders, layouts y eventos)
├── vision_worker.py  # Hilo de trabajo secundario (cámara, MediaPipe y filtro EMA)
└── styles.py         # Hoja de estilos con el tema oscuro moderno
```

- **Separación clara de tareas:** `main_gui.py` no toca píxeles de la cámara ni procesa matrices, solamente recibe las fotos ya listas y los ángulos calculados a través de señales (`Signal/Slot`). Si el día de mañana cambiamos MediaPipe por otro modelo de visión, la interfaz gráfica no tiene que modificarse.

---

## 7. Pruebas de Integración con el Hardware Físico

Esta tabla resume las pruebas físicas que se deben certificar en el banco de pruebas conectando la GUI directamente con el ESP32 del robot:

| Caso de Prueba | Acción en la GUI | Comportamiento Esperado en el Robot | Estado |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **P-01: Conexión Inalámbrica** | Dar clic en "Conectar BLE" con la MAC del robot | El botón pasa a color verde "Desconectar" y el ESP32 reporta enlace en su monitor serial. | `Validado` |
| **P-02: Movimiento Manual** | Mover el slider de Q1 a 75° | El servomotor horizontal se mueve de forma suave al ángulo solicitado. | `Validado` |
| **P-03: Botón de Pausa** | Dar clic en "Pausar" mientras seguimos la mano | El video sigue prendido con el letrero `PAUSADO` y los servomotores se quedan quietos en su último lugar. | `Validado` |
| **P-04: Activación de Saludo** | Hacer el gesto de "Hola" frente a la cámara | La GUI envía el comando `SALUDO` y el robot realiza la oscilación física de saludo sin trabarse. | `Validado` |

---

*Documento elaborado como memoria técnica de la interfaz gráfica de usuario para el Segundo Corte de LabDST.*